

09_middleware 大纲

1.中间件概述	1
1.1 什么是中间件	1
情况1：没有中间件的Agent架构：	1
情况2：添加中间件之后的Agent架构变为：	2
1.2 为什么需要中间件	3
1.3 中间件的分类	4
1.4 和模型供应商无关的内置中间件分类	5
类型1：成本与资源控制类	5
类型2：稳定性与容错保障类	6
类型3：安全与合规风控类	6
类型4：决策增强与智能编排类	6
类型5：执行能力扩展类	7
类型6：开发调试与测试辅助类	7
2、常用内置中间件的使用	7
2.1 SummarizationMiddleware中间件	7
2.1.1 参数说明	8
参数1：model —用于摘要的模型	8
参数2：trigger —摘要触发条件	8
参数3：keep —摘要时保留的原始消息	8
参数4：token_counter —统计token数量的函数	8
参数5：summary_prompt —摘要时的自定义提示词	8
参数6：trim_token_to_summarize —摘要时历史消息的最大token数	8
2.1.2 举例1 测试trigger、keep参数	9
2.1.3 举例2 测试summary_prompt参数	10
2.2 HumanInTheLoopMiddleware中间件	12
2.2.1 参数说明	12
参数1：interrupt_on —工具名和中断策略的映射	12
参数2：description_prefix —自定义中断描述	13
2.2.2 举例过程1：调用前中断	13
2.2.3 举例过程2：指明工具调用请求决策	19
2.3 PIIMiddleware中间件	22
2.3.1 参数说明	22
参数1：pii_type —检测的PII数据类型	22
参数2：strategy —处理PII信息的策略	22
参数3：detector —自定义PII检测函数或者正则表达式	22
参数4：apply_to_input —是否在调用模型前检测	23
参数5：apply_to_output —是否在模型调用后检测	23
参数6：apply_to_tool_results —是否在工具调用后检测其输出	23
2.3.2 举例1：使用内置检测器	23
2.3.3 举例2：自定义检测器/函数	24
2.4 TodoListMiddleware中间件	26

2.4.1 参数说明	26
2.4.2 案例设计	27
2.4.3 代码	27
1、环境准备	27
2、业务代码	28
3、调用及输出	30
4、分析	40
3、其它内置中间件	41
3.1 ModelCallLimitMiddleware中间件	41
举例1：整个会话限制-优雅退出	41
举例2：整个会话限制-抛异常	44
举例3：单次调用限制-优雅退出	48
举例4：单次调用限制-抛异常	53
3.2 ToolCallLimitMiddleware中间件	57
举例1：整个会话限制-优雅结束	57
举例2：整个会话限制-抛异常	60
案例3：单次调用限制-继续运行	64
3.3 ModelFallbackMiddleware中间件	69
举例1：实例设置models	69
举例2：create_agent中设置model	69
3.4 LLMTToolSelectorMiddleware中间件	70
举例1:max_tool=0,always_include=["get_weather"]	71
举例2:max_tool=0,always_include=["get_news"]	73
举例3:max_tool=0,always_include=["get_weather","get_news"]	75
举例4:max_tool=1,always_include=["get_weather"]	77
3.5 ToolRetryMiddleware中间件	79
举例1：带抖动	79
举例2：无抖动	81
3.6 ModelRetryMiddleware中间件	85
举例1：继续运行	85
举例2：抛异常	85
3.7 LLMTToolEmulator中间件	86
3.8 ContextEditingMiddleware中间件	89
1. 实验组-启用上下文编辑	89
2. 对照组-不裁剪上下文	90
3.9 FilesystemFileSearchMiddleware中间件	91
3.10 Shell tool中间件	94
3.11 Filesystem中间件	94
3.12 Subagent中间件	94
4、多个中间件组合及执行顺序	94
5、自定义中间件	96

5.1 什么是hook函数(钩子函数)	96
5.2 LangChain的hook函数分类	98
类型1: Node-style hooks(节点风格钩子)	98
类型2: Wrap-style hooks(包装风格钩子)	98
5.3 Node-style hooks函数用法	98
5.3.1 基本用法	98
1. 基于装饰器实现	98
2. 基于类实现	100
5.3.2 两种方法的统一	102
5.3.3 参数说明	103
5.3.4 返回值说明	104
5.3.5 装饰器参数: can_jump_to	104
1. 基于装饰器实现	104
2. 基于类实现	120
5.4 Wrap-style hooks函数用法	114
5.4.1 基本用法	114
1. wrap_model_call	126
① 基于装饰器实现	98
② 基于类实现	100
场景1: 重试逻辑	85
场景2: 响应缓存	117
场景3: 修改系统提示	118
2. wrap_tool_call	118
① 基于装饰器实现	98
② 基于类实现	100
5.4.2 两种方法的统一	122
5.4.3 参数说明	122
5.5 装饰器和类的选择	122
情况1: 中间件只用一个钩子函数, 推荐用装饰器, 需要多个钩子函数推荐类写法	122
1. 使用装饰器实现	123
2. 使用类实现	123
情况2: 复杂配置推荐用类实现	124
情况3: 跨项目复用推荐用类写法	125
5.6 hook函数执行顺序(重要)	126
1. 代码	126
2. 分析	128